

ASSIGNMENT REPORT

Multi-Client GUI Chat Application over Mininet

1. Objective

The objective of this assignment is to transition a CLI-based TCP chat application into an event-driven Desktop GUI application using Python's tkinter library. The system is deployed and tested within a simulated network topology inside Mininet, allowing concurrent client nodes (h2, h3, h4, h5) to communicate through a centralized server node (h1) over Port 5000.

2. System Architecture & Design Decisions

2.1 Multithreading Architecture

The client application runs on two concurrent threads to prevent freezing:

- **Main GUI Thread:** Manages visual rendering and user interaction event loops (`root.mainloop()`).
- **Background Receiver Thread (Daemon):** Runs a blocking network loop (`socket.recv()`) asynchronously to catch incoming packets without interrupting the GUI.

2.2 TCP Buffer Stream Parsing

Because TCP frequently merges consecutive packets into a single buffer segment (e.g., merging system notices with active client lists like [System]: Yuva joined...USERS:Vasavi,Sahil...), a smart stream splitter was implemented. The parser detects the "USERS:" identifier, splits the payload, prints the clean chat text, and extracts the comma-separated roster to refresh the sidebar dynamically.

2.3 UI Layout Decisions

- **Grid Layout:** Built using the `grid()` layout manager to allow clean, proportional scaling.
- **Read-Only History:** Uses a `scrolledtext.ScrolledText` widget toggled to `DISABLED` to maintain message history integrity.
- **Roster Sidebar:** Uses a standard `Listbox` to display real-time active users.

3. Reused Components from CLI Implementation

The application preserves verified structural elements from the previous command-line version:

- TCP socket backend configuration connecting to host `10.0.0.1` on port `5000`.
- Server-side multi-threading tracking client sockets mapped to usernames in a shared dictionary.
- Private message routing logic using the `/pm <username> <message>` parser.

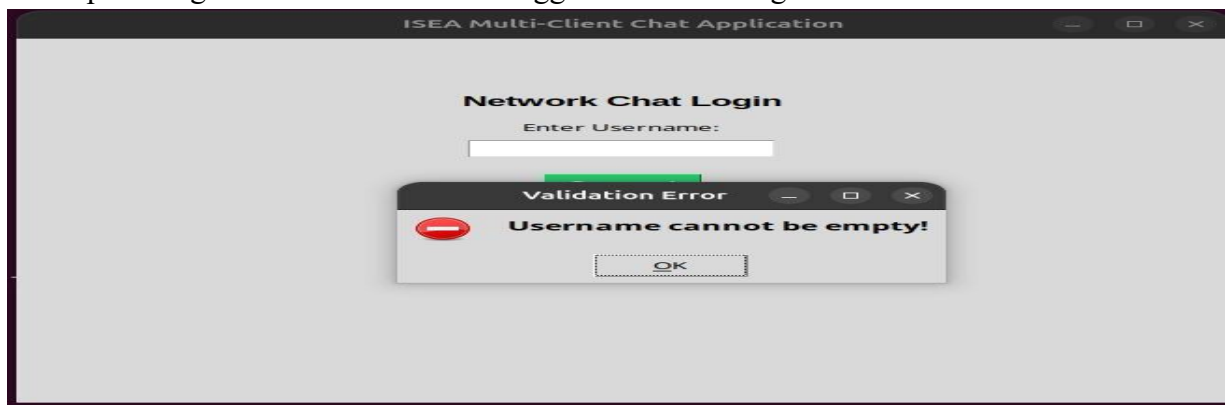
4. Experimental Setup & Network Topology

- **Mininet Command:** `sudo mn --topo single,5`
- **Server Node (h1):** `python3 server.py`
- **Client Nodes (h2 to h5):** `python3 client_gui.py`

5. Test Scenarios & Visual Verification

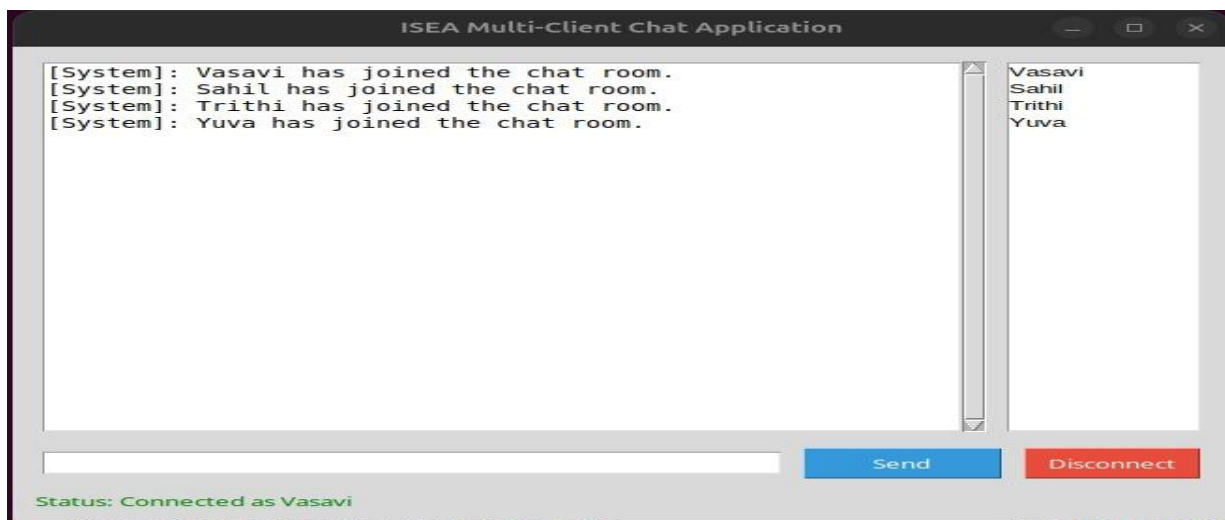
A. Username Validation Error

Attempts to log in without a username trigger an error dialogue.



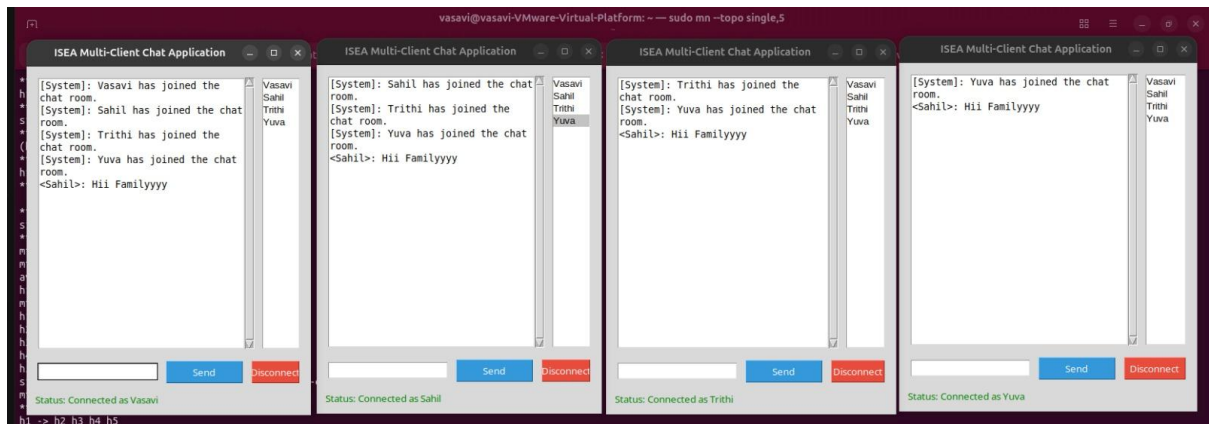
B. User Joining & Room Synchronization

Displays system notifications on all terminals when new users connect.



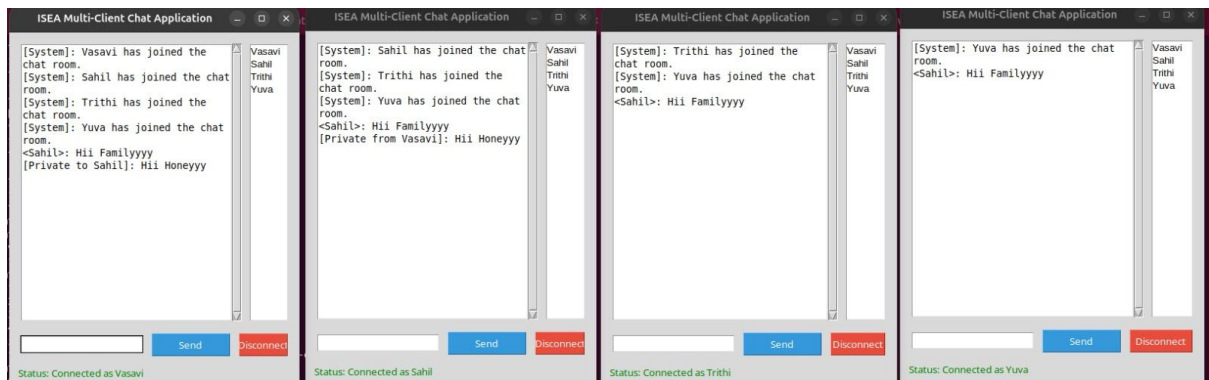
C. Broadcast Messaging

Verifies global message delivery.



D. Private Messaging

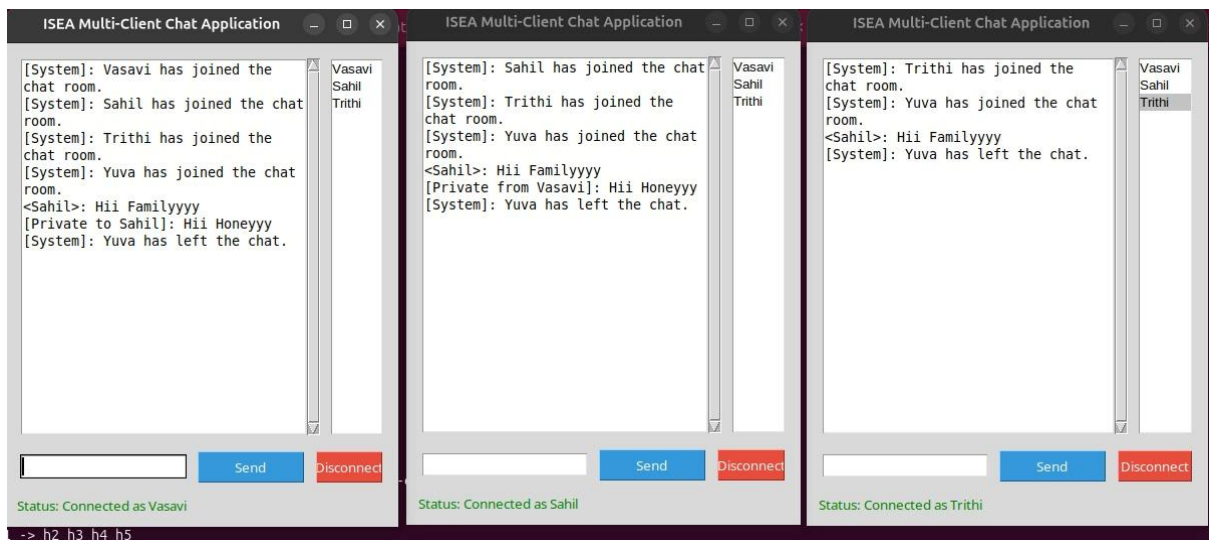
Ensures isolated, targeted messaging.



E. User Leaving & Disconnection

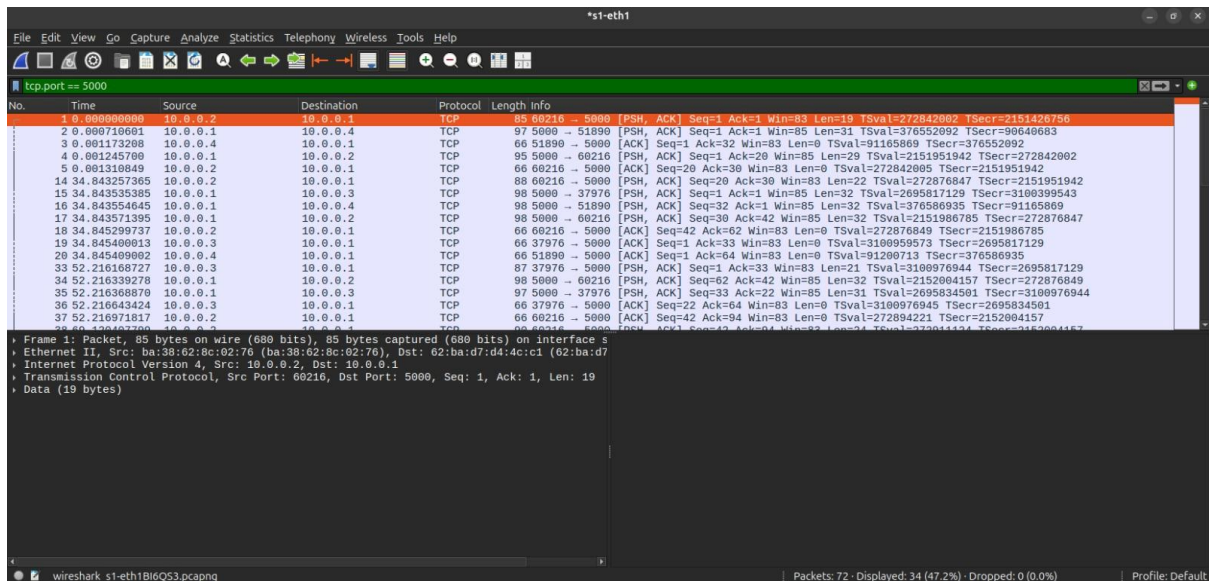
When a client disconnects, the server cleans up the connection, broadcasts a departure notice, and updates the online rosters of all remaining clients.

Validates dynamic client cleanup upon exit.



6. Wireshark Network Packet Analysis

A packet capture was performed on the loopback interface using Wireshark.



Packet Analysis:

- **Three-Way Handshake:** Clicking "Connect" initiates a connection. This is confirmed by the exchange of SYN → SYN-ACK → ACK flags between the client host and server 10.0.0.1.
- **Data Transmission:** When chat messages or roster updates are sent, packets carrying the PSH, ACK flags transfer the data over Port 5000.
- **Connection Teardown:** Clicking "Disconnect" cleanly closes the connection, confirmed by an exchange of FIN-ACK and ACK packets.

7. Reflection Answers

Q1. Why is it good practice to separate the GUI code from the socket networking code?

Answer: Separating the GUI presentation layer from the socket networking code enforces a clean Separation of Concerns (SoC) software architecture. Decoupling these layers ensures that the network communication protocol remains highly reusable. If the application needs to transition from a desktop framework (tkinter) to a web-based client or a command-line interface (CLI), the network logic can be imported directly with zero changes.

Q2. Why is a background thread required to handle receiving messages from the server? What would happen if we tried to run the receive loop in the main GUI thread?

Answer: Socket operations like `socket.recv()` are blocking operations—they stop thread execution until data is received over the network. The main GUI thread runs a continuous event loop (`root.mainloop()`) to render visual elements and process user input. If we ran the blocking `recv()` loop on the main thread, the entire GUI would freeze and become completely unresponsive.

unresponsive. Running these network operations on a background daemon thread keeps the user interface responsive at all times.

8. Conclusion

In this project, we designed and deployed a robust, multithreaded GUI chat application within a simulated Mininet environment. By handling TCP stream fragmentation and packet gluing, we ensured consistent message synchronization and dynamic roster updates. This experiment provided valuable hands-on experience with multi-threaded event-driven UI design, socket programming, and network traffic analysis using Wireshark.